

““latex

Fizikçiler İçin Bilgisayar Programlama

Kontrol Yapıları: Döngüler (for, while, do-while)

Fizik Bölümü

2 Ekim 2026

- 1 Giriş: Fizikte Döngü Mantığı
- 2 for Döngüsü
- 3 while Döngüsü
- 4 do-while Döngüsü
- 5 Döngü Karşılaştırması ve En İyi Pratikler

Fizikte karşılaştığımız problemlerin büyük bir kısmı analitik olarak kapalı formda çözülemez. Bu durumlarda sayısal (nümerik) yaklaşımlar devreye girer:

- **Zaman Evrimi:** Δt adımlarıyla parçacık yörüngeleri (Euler, Verlet, Runge-Kutta).
- **Seri Açılımları:** Taylor/Maclaurin serileri, Fourier toplamaları, özel fonksiyonlar ($J_n(x)$, $P_n(x)$).
- **Yakınsama Kriterleri:** İstenen hata payına (ϵ) ulaşana kadar iterasyon.
- **DeneySEL Veri Analizi:** Sensörlerden veya dedektörlerden okunan verilerin filtrelenmesi ve istatistiği.

Temel İlke

Döngüler, belirli bir koşul sağlandığı sürece kod bloklarını tekrarlı olarak çalıştıran yapılardır.

for Döngüsü: Yapı ve Akış

Genellikle adım sayısı veya sınırları önceden bilinen iterasyonlarda tercih edilir.

```
for (başlangıç; koşul; artış/azalış) { ... }
```

Bileşenler:

- 1 **Başlangıç:** Sayaç tanımlanır ve ilk değer atanır.
- 2 **Koşul:** Döngü gövdesine girmeden önce test edilir (doğruysa devam).
- 3 **Adım:** Gövde bittiğinde sayaç güncellenir.

Fiziksel Kullanım Alanı

Sabit grid aralıklarında ($x_i = x_0 + i\Delta x$), sabit boyutlu matris işlemlerinde ve sonlu toplam hesaplarında standart tercihtir.

for Örnek 1: Harmonik Seri ve Enerji Kuantumu Toplamı

Problem: İlk N terimin toplamını hesaplama: $S_N = \sum_{n=1}^N \frac{1}{n}$

Algoritma:

- 1 Toplam değişkenini ($S = 0.0$) sıfırla.
- 2 $n = 1$ 'den N 'ye kadar birer artan döngü kur.
- 3 Her adımda $S = S + 1.0/n$ güncellemesi yap.

```
1  #include <iostream>
2
3  int main() {
4      int N = 1000;
5      double toplam = 0.0;
6
7      for (int n = 1; n <= N; ++n) {
8          toplam += 1.0 / n;
9      }
10
11     std::cout << "N = " << N << " için Toplam: " << toplam << std::endl;
12     return 0;
13 }
14
```

for Örnek 2: Kinematik Tablolama (Serbest Düşme)

Problem: $h_0 = 100$ m yükseklikten bırakılan cismin $t = 0$ ile $t = 4$ s arasındaki konumunu $\Delta t = 0.5$ s adımlarla tablo haline getirme: $y(t) = h_0 - \frac{1}{2}gt^2$.

```
1  #include <iostream>
2  #include <iomanip>
3
4  int main() {
5      const double g = 9.80665;
6      const double h0 = 100.0;
7      const double dt = 0.5;
8
9      std::cout << "t (s)\t\t y (m)\n-----\n";
10     for (double t = 0.0; t <= 4.0 + 1e-6; t += dt) {
11         double y = h0 - 0.5 * g * t * t;
12         std::cout << std::fixed << std::setprecision(2)
13         << t << "\t\t" << y << std::endl;
14     }
15     return 0;
16 }
17
```

Not: Kayan noktalı artışlarda yuvarlama hatalarına karşı bitiş koşuluna tolerans ($\epsilon = 10^{-6}$) eklenir.

for Örnek 3: Sayısal İntegral (Sol Dikdörtgen Kuralı)

Problem: $W = \int_0^2 (3x^2 + 2x) dx$ iş integralini N dilime bölerek hesaplama.

```
1  #include <iostream>
2
3  double f(double x) { return 3.0 * x * x + 2.0 * x; }
4
5  int main() {
6      double a = 0.0, b = 2.0;
7      int N = 10000;
8      double dx = (b - a) / N;
9      double integral = 0.0;
10
11     for (int i = 0; i < N; ++i) {
12         double x_i = a + i * dx;
13         integral += f(x_i) * dx;
14     }
15
16     std::cout << "Hesaplanan Is: " << integral << " J (Analitik: 12.0 J)" << std::endl;
17     return 0;
18 }
19
```

while Döngüsü: Yapı ve Yakınsama Mantığı

Döngünün kaç kez çalışacağı önceden bilinmediğinde, yalnızca belirli bir şart devam ettiği sürece yürütülecek işlemlerde kullanılır.

```
while (koşul) { ... }
```

Kritik Nokta: Sonsuz Döngü Tehlikesi

Döngü gövdesi içinde koşulu değiştirecek bir mekanizma (sayaç artışı, hata farkı güncellemesi vb.) bulunmalıdır. Aksi takdirde program kilitlenir.

Fiziksel Senaryolar

- Bir parçacık zeminle temas edene kadar ($y > 0$).
- Ardışık Taylor terimleri makine hassasiyetinin altına inene kadar ($|a_n| > \epsilon$).
- Radyoaktif bozunumda çekirdek sayısı başlangıcın yarısına inene kadar.

while Örnek 1: Taylor Serisi ile e^x Yakınsaması

Problem: $e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$ serisini $|term| < 10^{-8}$ olana kadar toplama.

```
1  #include <iostream>
2  #include <cmath>
3
4  int main() {
5      double x = 1.5;
6      double term = 1.0;  // n = 0 terimi
7      double sum = 0.0;
8      int n = 0;
9      const double eps = 1e-8;
10
11     while (std::abs(term) > eps) {
12         sum += term;
13         n++;
14         term = term * (x / n); // 0(1) rekans
15     }
16
17     std::cout << "Hesap: " << sum << ", std::exp: " << std::exp(x)
18     << ", Adim sayisi: " << n << std::endl;
19     return 0;
20 }
21
```

while Örnek 2: Yarılanma Ömrü Simülasyonu

Problem: $N(t) = N_0 e^{-\lambda t}$ bozunumunda çekirdek sayısı $N_0/2$ 'ye ne zaman ulaşır? (Adım adım Euler yaklaşımı: $\Delta N = -\lambda N \Delta t$).

```
1  #include <iostream>
2
3  int main() {
4      double N0 = 1.0e6, N = N0;
5      double lambda = 0.05; // 1/saat
6      double t = 0.0, dt = 0.001;
7
8      while (N > N0 / 2.0) {
9          double dN = -lambda * N * dt;
10         N += dN;
11         t += dt;
12     }
13
14     std::cout << "Simulasyon T_1/2: " << t << " saat (Teorik: "
15     << 0.693147 / lambda << " saat)" << std::endl;
16     return 0;
17 }
18
```

while Örnek 3: Düşey Atış ve Yere Çarpma Süresi

Problem: $v_0 = 40 \text{ m/s}$ ile yukarı atılan cismin yere çarpma ($y \leq 0$) anını belirleme: $v_{k+1} = v_k - g\Delta t$,
 $y_{k+1} = y_k + v_{k+1}\Delta t$.

```
1  #include <iostream>
2
3  int main() {
4      const double g = 9.81;
5      double y = 0.0, v = 40.0;
6      double t = 0.0, dt = 0.001;
7
8      // İlk adimi baslatmak icin y >= 0 sartina dikkat edilir
9      t += dt; y += v * dt;
10     while (y > 0.0) {
11         v -= g * dt;
12         y += v * dt;
13         t += dt;
14     }
15
16     std::cout << "Ucus suresi: " << t << " s (Teorik 2*v0/g: "
17     << (2.0 * 40.0 / g) << " s)" << std::endl;
18     return 0;
19 }
20
```

do-while Döngüsü: Yapı ve Karakteristikleri

Şart ne olursa olsun ****en az bir kez**** çalışması garanti olan döngü yapısıdır. Koşul kontrolü blok sonunda yapılır.

```
do { ... } while (koşul);
```

Tipik Kullanım Durumları:

- Kullanıcıdan parametre alma ve giriş doğrulama (ör. kütle $m \leq 0$ girilemez).
- İlk adımın mutlak surette atılması gereken dinamik hesaplar.
- Menü arayüzleri.

Sözdizimi Hatırlatması

Diğer döngülerin aksine, do-while yapısının sonundaki parantezden sonra noktalı virgül (;) konulması zorunludur.

do-while Örnek 1: Fiziksel Parametre Doğrulama

Problem: Kullanıcıdan kesinlikle pozitif olması gereken bir mutlak sıcaklık ($T > 0\text{ K}$) verisi alma.

```
1  #include <iostream>
2
3  int main() {
4      double T;
5      do {
6          std::cout << "Sicaklik degerini girin (Kelvin, T > 0): ";
7          std::cin >> T;
8          if (T <= 0.0) {
9              std::cout << "Hata: Termodinamik sicaklik sifir veya negatif olamaz!\n";
10             }
11     } while (T <= 0.0);
12
13     std::cout << "Gecerli sicaklik: " << T << " K\n";
14     return 0;
15 }
16
```

do-while Örnek 2: Newton-Raphson ile Kök Bulma

Problem: $f(x) = x^2 - 2 = 0$ denkleminin kökünü ($\sqrt{2}$) Newton-Raphson iterasyonu ile bulma:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{x_k^2 - 2}{2x_k}.$$

```
1  #include <iostream>
2  #include <cmath>
3
4  int main() {
5      double x = 1.0; // Baslangic tahmini
6      double x_next;
7      const double eps = 1e-9;
8      int iter = 0;
9
10     do {
11         x_next = x - (x * x - 2.0) / (2.0 * x);
12         double delta = std::abs(x_next - x);
13         x = x_next;
14         iter++;
15         if (delta < eps) break;
16     } while (true);
17
18     std::cout << "Karekok(2) tahmini: " << x << " (" << iter << " iterasyon)" << std::endl;
19     return 0;
20 }
21
```

do-while Örnek 3: Sönümlü Salıncı Simülasyonu

Problem: $m\ddot{x} + \gamma\dot{x} + kx = 0$ harmonik hareketinde, sistem durma noktasına ($|E_{mekanik}| < E_{esik}$) gelene kadar simülasyonu en az bir döngü adımı yürütme.

```
1  #include <iostream>
2
3  int main() {
4      double x = 1.0, v = 0.0;
5      const double k = 10.0, m = 1.0, gamma = 0.2;
6      const double dt = 0.01;
7      double E_total;
8      int step = 0;
9
10     do {
11         double F = -k * x - gamma * v;
12         double a = F / m;
13         v += a * dt;
14         x += v * dt;
15         E_total = 0.5 * m * v * v + 0.5 * k * x * x;
16         step++;
17     } while (E_total > 1e-4 && step < 100000);
18
19     std::cout << "Sistem " << step * dt << " saniye sonra sonuqlendi." << std::endl;
20     return 0;
21 }
22
```

Hangi Döngü Ne Zaman Seçilmeli?

Döngü Türü	Koşul Denetimi	Min. Çalışma	Tipik Fizik Problemi
for	Başta (Pre-test)	0	Matris/Dizi, Sabit Δt tablolama, İntegral
while	Başta (Pre-test)	0	Yakınsama testleri, Taylor serisi, Bozunum
do-while	Sonda (Post-test)	1	Newton-Raphson, Doğrulama, Eşik sönümü

Önemli Notlar:

- Kayan noktalı sayaçlarda ('float'/'double') eşitlik ('==') kontrolü yapılmamalı, eşitsizlik ('<=' veya fark ϵ) kullanılmalıdır.
- Döngü içinde matris veya büyük bellek tahsisinden kaçınılmalı, dinamik tahsisler döngü dışında yapılmalıdır.